

Milestone_3_Web_Scraping_and_Data_Collection

May 29, 2026

1 Milestone3: Web Scraping and Data Collection

Cleaning/Formatting Website Data

Perform at least 5 data transformation and/or cleansing steps to your website data. The below examples are not required - they are just potential transformations you could do. If your data doesn't work for these scenarios, complete different transformations. You can do the same transformation multiple times if needed to clean your data. The goal is a clean dataset at the end of the milestone. As a reminder - you cannot export your website data to CSV to work with it, you must do all the work directly against the HTML source.

Examples:

- Replace Headers.
- Format data into a more readable format.
- Identify outliers and bad data.
- Find duplicates.
- Fix casing or inconsistent values.
- Conduct Fuzzy Matching.

Make sure you clearly number and label each transformation step (Step #1, Step #2, etc.) in your code and describe what it is doing in 1-2 sentences.

```
[1]: # Load the necessary libraries.  
# import pandas as pd  
# import numpy as np  
# from scipy import stats
```

```
from bs4 import BeautifulSoup  
import requests
```

```
[2]: # Define headers to mimic a browser request.  
headers = {  
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36_  
↳(KHTML, like Gecko) Chrome/130.0.0.0 Safari/537.36'  
}
```

```
[3]: ''' Previously used working code when originally written
```

```

try:
    # Send a GET request to the IMDB Top 250 movies page.
    source = requests.get('https://www.imdb.com/chart/top/?ref_=nv_mv_250',
    ↪headers = headers)

    # Raise an error for unsuccessful requests.
    source.raise_for_status()

    # Parse the HTML content using BeautifulSoup.
    soup = BeautifulSoup(source.text, 'html.parser')

    # Find the list of movies in the HTML structure.
    # movies = soup.find('ul', role = 'presentation').find_all('li',
    ↪class_='ipc-metadata-list-summary-item sc-4929eaf6-0 DLYcv cli-parent')
    movies = soup.find('ul', role = 'presentation').find_all('li',
    ↪class_='ipc-metadata-list ipc-metadata-list--dividers-none sc-4b25d7ad-0
    ↪fouuLN alt_item_colors compact-list-view base ipc-metadata-list--base')

    movie_list = soup.find('ul', role='presentation')

    if movie_list is None:
        print("Movie list was not found. IMDb may have changed the page or
    ↪blocked the request.")
    else:
        movies = movie_list.find_all('li',
    ↪class_='ipc-metadata-list-summary-item')

    # Iterate through each movie item,
    for movie in movies:

        # Extract the movie name.
        name = movie.find('h3', class_='ipc-title__text').text

        # Extract year, duration, and rating from the metadata div.
        year_duration_rating = movie.find('div', class_='sc-732ea2d-5 kHnTQb
    ↪cli-title-metadata').text

        # Extracting year, duration, and rating from the combined string.
        year = year_duration_rating[:4] # First 4 characters are the year.
        duration = year_duration_rating[4:-1] # Everything between the year
    ↪and the last character.
        rating = year_duration_rating[-1] # Last character is the rating.

        # Format duration to separate hours and minutes for readability.
        duration = duration.replace('h', 'h ').replace('m', 'm ')

```

```

        # Extract the star rating and vote count.
        star_rating = movie.find('span', class_='ipc-rating-star--rating').text
        vote_count = movie.find('span', class_='ipc-rating-star--voteCount').
↪text

        # Clean the vote count by removing parentheses and extra spaces.
        vote_count_cleaned = vote_count.replace('(', '').replace(')', '').
↪strip()

        # Print the extracted movie details in a formatted way. Preferred way.
        print(name, year, duration, rating, star_rating, vote_count_cleaned)

        # Print the extracted movie details in a formatted way. Option #2.
        # print(f'Name: {name}')
        # print(f'Year: {year}')
        # print(f'Duration: {duration}')
        # print(f'Rating: {rating}')
        # print(f'Star Rating: {star_rating}')
        # print(f'Vote Count: {vote_count_cleaned}')
        # print('---') # Separator for readability

        # Print the extracted movie details in a formatted way. Option #3.
        # print(f'Name: {name}\nYear: {year}\nDuration: {duration}\nRating:
↪{rating}\nStar Rating: {star_rating}\nVote Count: {vote_count.replace("(",
↪"".replace(")", "").strip()})')
        # print('---') # Separator for readability

except Exception as e:
    # Print any exceptions that occur during the request or parsing.
    print(e)
'''

try:
    source = requests.get("https://www.imdb.com/chart/top/?ref_=nv_mv_250",
↪headers=headers)

    print("Status code:", source.status_code)

    soup = BeautifulSoup(source.text, "html.parser")

    movie_list = soup.find("ul", role="presentation")

    if movie_list is None:
        print("Movie list was not found.")

```

```

        print("IMDb is likely blocking the request, so no movie data was_
↳scraped.")
        movies = []
    else:
        movies = movie_list.find_all("li",_
↳class_="ipc-metadata-list-summary-item")
        print(f"Movies found: {len(movies)}")

    for movie in movies:
        name = movie.find("h3", class_="ipc-title__text")
        rating = movie.find("span", class_="ipc-rating-star--rating")
        vote_count = movie.find("span", class_="ipc-rating-star--voteCount")

        print(
            name.text if name else "N/A",
            rating.text if rating else "N/A",
            vote_count.text if vote_count else "N/A"
        )

except Exception as e:
    print("Error:", e)

```

Status code: 202

Movie list was not found.

IMDb is likely blocking the request, so no movie data was scraped.

2 Ethical implications of data wrangling specific to my data-sources.

In this project, I scraped data from the IMDb.com website. Web scraping raises ethical concerns, especially if the site's terms of service are violated or if confidential data is unintentionally accessed. I ensured that all data was publicly accessible and no personal information was involved. Transforming the data could introduce bias, particularly when assumptions are made to fill in missing values or correct inconsistencies. I commented on each web scrapping step to mitigate these risks and referenced the websites' policies to confirm ethical data use. Also, I minimized the risk of misrepresenting the data or drawing flawed conclusions.

IMDb returned a protected webpage instead of the normal Top 250 movie list. Because of this, the movie list could not be scraped using requests and BeautifulSoup. This shows one limitation of web scraping: websites can change their structure or block automated requests.